ESCENARIOS DE PRUEBA QA – GRUPO 2

API y Seguridad | Checklist de seguimiento por prioridad

#	Prioridad	ID	Área	Escenario de prueba	Resultado esperado	Seguimiento / Notas	Estado	Evidencias
1	**ALTA**	G2-01	Login	Iniciar sesión con usuario y contraseña correctos	Permite acceso y crea sesión/token válido	Se realizó una solicitud POST /api/v1/auth/login utilizando credenciales válidas de la cuenta administradora QA. La API respondió HTTP 200 OK, generando un access_token de tipo bearer, con una vigencia declarada de 7200 segundos y los datos correspondientes al usuario autenticado. Posteriormente, se utilizó el JWT obtenido para autorizar una solicitud GET /api/v1/me. El endpoint respondió HTTP 200 OK y devolvió correctamente la información del administrador QA (ID 9), incluyendo su rol admin, estado activo y permisos asociados.Se confirmó que el inicio de sesión genera un JWT funcional y permite acceder a un endpoint protegido, completando la validación que había quedado pendiente en la evaluación original. El retest funcional se realizó con la cuenta administradora QA; se conservan como antecedente las seis pruebas de inicio de sesión exitosas documentadas anteriormente.	PASS	Evidencias
2	**ALTA**	G2-02	Login	Iniciar sesión con contraseña incorrecta	Rechaza acceso con 401	Se realizó una solicitud utilizando el correo de la cuenta administradora QA y una contraseña deliberadamente incorrecta. La API respondió HTTP 401 Unauthorized, con código de error UNAUTHORIZED y el mensaje genérico "Correo o contraseña incorrectos". La respuesta mantuvo la estructura JSON uniforme del sistema y no incluyó ningún access_token, impidiendo el inicio de sesión.Se confirmó que el sistema continúa rechazando correctamente las contraseñas incorrectas, manteniendo el comportamiento esperado y previamente documentado.	PASS	Evidencias
3	**ALTA**	G2-05	Sesión	Usar sesión/token después de cerrar sesión	El acceso debe ser rechazado	Se verificó POST /api/v1/auth/logout con JWT válido de docente, obteniendo HTTP 200 y el mensaje «Sesión cerrada». Posteriormente, se reutilizó el mismo JWT en GET /api/v1/me, obteniendo HTTP 401 con el mensaje «La sesión no existe o ya fue cerrada». Se confirma la revocación efectiva del token tras el cierre de sesión.	PASS	Evidencias
4	**ALTA**	G2-06	Autenticación	Acceder a un endpoint protegido sin iniciar sesión	Rechaza acceso con 401	se verificó la protección de endpoints mediante autenticación JWT utilizando Swagger. Se ejecutó GET /api/v1/me sin autenticación, obteniendo HTTP 401 (UNAUTHORIZED), y posteriormente con un JWT válido de la cuenta docente, obteniendo HTTP 200 con los datos y el rol correspondientes al usuario autenticado. Adicionalmente, se ejecutó GET /api/v1/users sin enviar credenciales ni encabezados de autenticación, obteniendo HTTP 401 con el mensaje "Falta el token de autenticación", sin exponer información protegida. Se confirma la corrección del hallazgo original, en el que GET /api/v1/users respondía HTTP 422 por ausencia del encabezado usuario-actual en lugar del HTTP 401 esperado. El escenario cumple el comportamiento esperado dentro del alcance evaluado.	PASS	Evidencias

5	**ALTA**	G2-08	Permisos	Usuario autenticado intenta ejecutar una acción para la que no tiene permiso	Rechaza con 403	se verificó el control de permisos del backend mediante Swagger. Se inició sesión con la cuenta QA docente (rol teacher), obteniendo HTTP 200 y un JWT válido. Posteriormente, se utilizó dicho token para ejecutar GET /api/v1/users, endpoint reservado para administradores. La API respondió HTTP 403 (FORBIDDEN), con el mensaje "No tienes permisos necesarios para realizar esta acción", sin devolver información protegida. Se confirma que el backend aplica correctamente la restricción de permisos en el endpoint evaluado y que se resolvió el problema de integración JWT que anteriormente impedía completar la prueba, cuando se obtenía HTTP 422. El escenario cumple el resultado esperado dentro del alcance probado..	PASS	Evidencias
6	**ALTA**	G2-09	Roles	Docente intenta acceder a funciones exclusivas de administrador	Acceso rechazado	se verificó la restricción de acceso a funciones administrativas utilizando una cuenta autenticada con rol docente (teacher). Se inició sesión mediante POST /api/v1/auth/login, obteniendo HTTP 200 y un JWT válido. Posteriormente, se ejecutó GET /api/v1/users utilizando dicho token, con el objetivo de comprobar si un docente podía acceder a una función reservada para administradores. La API respondió HTTP 403 (FORBIDDEN), con el mensaje "No tienes permisos necesarios para realizar esta acción", sin exponer información de los usuarios. Se confirma la corrección del problema de integración JWT identificado en la evaluación original, cuando el endpoint respondía HTTP 422 e impedía verificar la restricción de acceso. El escenario cumple el comportamiento esperado dentro del alcance probado. Se reutilizó la prueba funcional de G2-08, debido a que ambos escenarios evalúan el mismo endpoint y la misma restricción de acceso	PASS	Evidencias
7	**ALTA**	G2-10	Roles / Backend	Saltarse el frontend e intentar ejecutar directamente una acción prohibida	El backend rechaza la operación	Se verificó que GET /api/v1/users devuelve HTTP 401 sin JWT, incluso con el encabezado usuario-actual falsificado. Con JWT válido de docente devuelve HTTP 403, tanto sin encabezado como al intentar suplantar a un administrador. Se confirma la corrección de la vulnerabilidad identificada inicialmente en G2-10.	PASS	Evidencias

8	**ALTA**	G2-15	Seguridad XSS	Introducir contenido con código/script en campos de texto	El código no se ejecuta en ninguna pantalla	se realizó una prueba de Cross-Site Scripting (XSS) almacenado utilizando el usuario QA con ID 12. Mediante PATCH /api/v1/users/12 se introdujo la cadena <script>alert('QA-XSS')</script> en el campo nombres, obteniendo HTTP 200. Posteriormente, GET /api/v1/users/12 confirmó que el contenido permanecía almacenado. Se revisó el archivo frontend-login/script.js y se identificó que el nombre del usuario se muestra mediante alert() después de un inicio de sesión exitoso. Al iniciar sesión con la cuenta de prueba, el navegador mostró la cadena introducida como texto literal dentro del mensaje de bienvenida, sin observarse la ejecución del script. Finalmente, se restauró el campo nombres a su valor original "QA" mediante PATCH, obteniendo HTTP 200.	PASS (caso evaluado)	Evidencias
9	**ALTA**	G2-16	Seguridad SQL	Probar entradas orientadas a SQL Injection	La entrada no altera consultas ni la base de datos	se realizó una prueba de SQL Injection en POST /api/v1/auth/login, introduciendo la cadena ' OR '1'='1 en el campo correo. La API respondió HTTP 401 (Unauthorized), con el código UNAUTHORIZED y el mensaje «Correo o contraseña incorrectos», sin generar un token de acceso ni permitir la autenticación. Posteriormente, se ejecutó GET /api/v1/users con credenciales administrativas válidas, obteniendo HTTP 200 y recuperando correctamente los registros de usuarios. Adicionalmente, se revisó el código de g2_api/routers/auth.py, confirmando que la búsqueda del usuario utiliza filtros de SQLAlchemy mediante Usuario.correo == datos.correo, sin concatenación directa de la entrada en sentencias SQL. También se verificó el uso de filtros de SQLAlchemy en la consulta de sesiones. El escenario se considera PASS (caso evaluado), ya que el intento de inyección no permitió evadir la autenticación y las consultas posteriores funcionaron correctamente. No se realizó una comparación de los registros antes y después de la prueba, por lo que no puede confirmarse de manera exhaustiva la ausencia de modificaciones en la base de datos. El resultado se limita al vector de inyección y a los componentes revisados.	PASS (caso evaluado)	Evidencias

10	**ALTA**	G2-17	Autorización	Cambiar un identificador para intentar acceder a recursos de otro usuario	El servidor verifica autorización y bloquea el acceso	se evaluó el control de autorización sobre recursos de otros usuarios mediante el endpoint PATCH /api/v1/users/{usuario_id}, utilizando Swagger y autenticación JWT. Inicialmente, se ejecutó una solicitud PATCH /api/v1/users/999999 con un JWT válido de la cuenta QA docente (ID 11), obteniendo HTTP 403 (FORBIDDEN), en lugar de permitir el procesamiento de la operación sin verificar los permisos. Posteriormente, se realizó una prueba de acceso horizontal y vertical utilizando el mismo JWT docente para intentar modificar la cuenta QA administradora (ID 9) mediante PATCH /api/v1/users/9, enviando el campo nombres con el valor "Prueba QA". La API respondió HTTP 403 (FORBIDDEN), con el mensaje "No tienes permisos necesarios para realizar esta acción". Finalmente, se inició sesión como administrador y se consultó GET /api/v1/me, obteniendo HTTP 200. Se verificó que el nombre del administrador permanecía como "QA", confirmando que el intento de modificación no alteró sus datos. Se confirma la corrección del hallazgo original relacionado con el procesamiento de solicitudes sin autenticación JWT en la ruta administrativa evaluada. El backend rechazó los intentos de modificación no autorizados y preservó la integridad de los datos del usuario objetivo.	PASS	Evidencias
11	**ALTA**	G2-19	Archivos	Intentar subir un tipo de archivo no permitido	Archivo rechazado	Se verificó que POST /api/v1/files rechaza archivos no permitidos. Un archivo TXT original obtuvo HTTP 400. Posteriormente, se modificó únicamente su extensión a .pdf, manteniendo su contenido de texto plano; la API volvió a responder HTTP 400 con código BAD_REQUEST. Se confirma el rechazo de ambos casos y la validación del contenido en el caso probado.	PASS	Evidencias
12	**ALTA**	G2-20	Archivos	Intentar subir un archivo que exceda el tamaño máximo	Archivo rechazado	Durante el retest del 23/10/2026 se confirmó que POST /api/v1/files implementa una restricción de tamaño máximo de 5 MB. Se verificó en el código fuente la constante MAX_SIZE = 5 * 1024 * 1024 y su validación. Al intentar cargar un archivo de 6 MB, la API respondió HTTP 400 Bad Request con el mensaje "El archivo supera el límite de 5 MB". El resultado coincide con el comportamiento esperado. Escenario aprobado.	PASS	Evidencias
13	**ALTA**	G2-21	Archivos / Permisos	Intentar descargar o acceder al archivo de otro usuario sin autorización	Acceso rechazado	Durante el retest del 23/10/2026 se detectó una falla de autorización en GET /api/v1/files/{id}. Se inició sesión con una cuenta docente y se solicitó un archivo previamente cargado por un administrador. La API respondió HTTP 200 y permitió su descarga, cuando el resultado esperado era rechazar el acceso. La revisión de files.py confirmó que se exige autenticación, pero no se verifica la propiedad del archivo ni los permisos específicos del usuario. Posible vulnerabilidad IDOR. Se requiere corregir el control de acceso y repetir la prueba.	FAIL	Evidencias

14	**ALTA**	G2-22	Archivos / Seguridad	Intentar manipular el nombre o ruta del archivo	No permite acceder ni escribir fuera del almacenamiento autorizado	Durante el retest se realizaron intentos de acceso mediante rutas manipuladas (../main.py y C:\Windows\win.ini), obteniendo HTTP 404 sin exposición de archivos internos. Adicionalmente, se revisó files.py y se verificó que los archivos se almacenan en la carpeta uploads con un nombre interno generado mediante UUID y extensión controlada por el tipo MIME permitido, sin utilizar directamente el nombre proporcionado por el usuario. El escenario se considera aprobado dentro del alcance evaluado.	PASS	Evidencias
15	**ALTA**	G2-23	Errores / Seguridad	Provocar errores para buscar información interna	No muestra SQL, rutas internas, stack traces ni secretos	se realizaron dos pruebas de manejo de errores en POST /api/v1/auth/login para verificar que las respuestas no expongan información interna sensible. En la primera prueba se envió un JSON mal formado, obteniendo HTTP 422 (Unprocessable Content), con el código VALIDATION_ERROR y un detalle controlado de tipo json_invalid. En la segunda prueba se enviaron valores numéricos en los campos correo y password, obteniendo nuevamente HTTP 422, con errores string_type para ambos campos. En las respuestas evaluadas no se observaron consultas SQL, rutas internas del servidor, stack traces, contraseñas, tokens ni otros secretos. La API devolvió mensajes de error estructurados mediante los campos error, code, message y details.	PASS (casos evaluados)	Evidencia
16	**ALTA**	G2-24	Información sensible	Revisar respuestas de API buscando información sensible	No expone contraseñas, secretos ni información sensible innecesaria	Al realizar una solicitud autenticada con un JWT de administrador, la API respondió HTTP 200 y devolvió una lista de usuarios con los campos id, correo, nombres, apellidos, activo y roles. No se observaron contraseñas, hashes, tokens ni claves secretas en los registros revisados. Adicionalmente, se inspeccionó la función diccionario() del archivo g2_api/routers/users.py, confirmando que construye las respuestas únicamente con los seis campos mencionados y excluye información sensible como hash_contrasena, password, access_token y refresh_token. Finalmente, se realizó una solicitud GET /api/v1/users	PASS	Evidencias
17	**ALTA**	G2-35	Configuración	Revisar repositorio y configuración	No contiene contraseñas ni variables sensibles	Se verificó que el archivo .gitignore contiene reglas para excluir archivos .env, entornos virtuales, bases de datos locales, archivos de registro, directorios de carga y otros archivos potencialmente sensibles. Asimismo, se revisó .env.example y se confirmó que utiliza valores de ejemplo para DATABASE_URL y SECRET_KEY, sin identificar credenciales reales en la plantilla. Posteriormente, se inspeccionó en GitHub la rama grupo-2 del repositorio UDV_Proyecto, específicamente la carpeta grupos/grupo-2-api-seguridad. No se observaron archivos .env, bases de datos locales ni directorios de carga publicados. Adicionalmente, se utilizó el buscador de archivos de GitHub para localizar archivos relacionados con .env en la rama evaluada. La búsqueda mostró únicamente dos archivos .env.example, uno en la raíz del repositorio y otro en la carpeta del Grupo 2.	PASS	Evidencias

18	**ALTA**	G2-37	Autenticación / Seguridad	Usar un token vencido para acceder a un endpoint protegido	Devuelve 401	Se revisó la función verificar_token() en g2_api/seguridad.py, confirmando que utiliza jwt. decode() para validar la firma y expiración del token y rechaza los tokens inválidos o expirados. Asimismo, se comprobó su integración con obtener_sesion_actual() en el módulo de autenticación. Posteriormente, se generó un JWT de prueba correctamente firmado, con una fecha de expiración anterior a la actual. La validación local rechazó el token y, al utilizarlo para acceder al endpoint protegido GET /api/v1/me, la API respondió HTTP 401 Unauthorized, con código UNAUTHORIZED y mensaje "Token inválido o vencido".	PASS	Evidencias
19	**ALTA**	G2-38	Autenticación / Seguridad	Usar un token modificado manualmente para acceder a un endpoint protegido	Devuelve 401	Inicialmente, se revisó la prueba automatizada test_token_alterado_no_es_valido, ubicada en tests/test_auth.py, y se ejecutó mediante pytest, obteniendo un resultado satisfactorio (1 passed). Posteriormente, se generó un JWT firmado con la configuración local del proyecto y se modificó manualmente un carácter de su firma sin volver a firmarlo. Al utilizar el token alterado para acceder al endpoint protegido GET /api/v1/me, la API respondió HTTP 401 Unauthorized, con código de error UNAUTHORIZED y mensaje "Token inválido o vencido". Se confirmó que el endpoint evaluado rechaza correctamente un JWT cuya firma ha sido modificada, resolviendo el bloqueo identificado en la evaluación original. La comprobación funcional se realizó sobre GET /api/v1/me.	PASS — JWT alterado rechazado correctamente	Evidencias
20	**MEDIA**	G2-03	Login	Iniciar sesión con usuario inexistente	Acceso rechazado sin revelar información sensible	se realizó una solicitud POST /api/v1/auth/login mediante Swagger, utilizando un correo no registrado y una contraseña de prueba. La API respondió HTTP 401 (Unauthorized), con el código UNAUTHORIZED y el mensaje genérico «Correo o contraseña incorrectos», sin generar un token de acceso ni revelar información sobre la existencia del usuario. Posteriormente, se repitió la prueba desde el formulario de inicio de sesión del frontend en http://localhost:5500, utilizando las mismas credenciales. El sistema mostró el mensaje «Correo o contraseña incorrectos» y rechazó el inicio de sesión.Se confirmó que tanto la API como el frontend rechazan el acceso con un usuario inexistente sin revelar información sensible sobre la existencia de la cuenta.	PASS	Evidencias
21	**MEDIA**	G2-04	Login / Validación	Enviar usuario y contraseña vacíos	Solicitud rechazada mediante validación	Se realizaron pruebas de inicio de sesión con campos vacíos tanto en la API como en el frontend. En la API, al enviar correo y contraseña vacíos, se obtuvo HTTP 401 (Unauthorized), rechazando el acceso. Posteriormente, desde http://localhost:5500, se comprobó que el formulario identifica los campos obligatorios: al dejar Usuario vacío, solicita completarlo; al ingresar un correo y dejar Contraseña vacía, muestra la misma validación. En ambos casos, el navegador impide enviar el formulario. Se cumple el resultado esperado de rechazar el inicio de sesión con campos vacíos. Como observación, la API devuelve 401 en lugar de un error de validación 422.	PASS	Evidencias

22	**MEDIA**	G2-07	Identidad	Consultar /me con un usuario autenticado	Devuelve usuario, roles y permisos correspondientes	Se verificó el funcionamiento de GET /api/v1/me. Sin token JWT devuelve HTTP 401. Con JWT válido de docente devuelve la identidad correcta (ID 11), correo, estado activo, rol teacher y permisos asociados. Se confirma la implementación del endpoint anteriormente no disponible.	PASS	Evidencia
23	**MEDIA**	G2-11	Usuarios	Consultar un usuario existente	Devuelve la información permitida correctamente	Se realizó la consulta sin autenticación JWT válida, obteniendo HTTP 401 (Unauthorized), lo que confirmó que el endpoint exige autenticación.Posteriormente, se repitió la solicitud utilizando un token JWT de administrador. La API respondió HTTP 200 (OK) y devolvió correctamente la información del usuario solicitado, incluyendo únicamente los campos id, correo, nombres, apellidos, activo y roles, sin exponer contraseñas, hashes ni tokens. Adicionalmente, se realizó la misma consulta utilizando un token JWT correspondiente a un usuario con rol docente. La API respondió HTTP 403 (Forbidden), rechazando el acceso por falta de permisos administrativos. En comparación con la evaluación original, se confirmó que actualmente existe el endpoint de consulta individual y que se aplican controles de autenticación y autorización basados en JWT y roles.	PASS — Retest satis	Evidencias
24	**MEDIA**	G2-12	Usuarios	Consultar un ID de usuario inexistente	Responde 404	Se verificó GET /api/v1/users/{usuario_id} con JWT de administrador. La consulta del usuario existente (ID 11) devolvió HTTP 200 con los datos correctos, sin exponer contraseñas ni hashes. La consulta del ID inexistente 999999 devolvió HTTP 404, código NOT_FOUND y mensaje descriptivo. Se confirma la implementación y el manejo adecuado de usuarios inexistentes.	PASS	Evidencias
25	**MEDIA**	G2-13	Validación	Enviar datos obligatorios vacíos	Responde 422 indicando los campos inválidos	Se ejecutó una solicitud incluyendo únicamente el campo correo y omitiendo el campo obligatorio password. La API respondió HTTP 422 (Unprocessable Content), rechazando la solicitud por incumplimiento de los requisitos de validación. Posteriormente, se realizó una segunda solicitud incluyendo únicamente el campo password y omitiendo correo. La API respondió HTTP 422, identificando correctamente el campo correo mediante el mensaje "Field required" y el tipo de error "missing". Los resultados confirman que la API rechaza solicitudes incompletas e identifica los campos obligatorios ausentes. Se mantiene el comportamiento esperado de validación observado en la evaluación original.	PASS	Evidencias
26	**MEDIA**	G2-14	Validación	Enviar datos con formato incorrecto	No procesa ni almacena información inválida	Se realizó una solicitud enviando el campo correo con un valor numérico (12345) en lugar de una cadena de texto. La API respondió HTTP 422 (Unprocessable Content), rechazando la solicitud con el mensaje "Input should be a valid string". Posteriormente, se ejecutó una segunda solicitud enviando el campo password con un valor numérico (12345678). La API respondió nuevamente HTTP 422, con código VALIDATION_ERROR, identificando el campo password mediante el mensaje "Input should be a valid string" y el tipo de error string_type. Ambas pruebas confirmaron que el backend identifica y rechaza tipos de datos incompatibles antes de completar el proceso de autenticación. No se verificó directamente la persistencia de datos en la base de datos, por lo que la conclusión se limita al rechazo de las solicitudes inválidas.	PASS	Evidencias

27	**MEDIA**	G2-18	Archivos	Subir un tipo de archivo permitido	Archivo aceptado y devuelve file_id	Se verificó la carga de un archivo PDF válido mediante POST /api/v1/files, obteniendo HTTP 200 y un file_id único. Posteriormente, se recuperó el archivo mediante GET /api/v1/files/{id}, obteniendo HTTP 200 y confirmando que el PDF descargado corresponde al original. Se observó una inconsistencia en Swagger: el identificador se envía como parámetro query (file_id) en lugar de sustituir el parámetro de ruta {id}. Se recomienda revisar el contrato del endpoint.	PASS	Evidencias
28	**MEDIA**	G2-25	API / HTTP	Realizar correctamente una solicitud de creación	Utiliza el código HTTP correspondiente, por ejemplo 201	Se intentó crear un usuario utilizando un token JWT correspondiente al rol docente. La API respondió HTTP 403 (Forbidden), rechazando correctamente la operación por falta de permisos administrativos. Posteriormente, se repitió la solicitud utilizando un token JWT de administrador y datos válidos de un usuario ficticio. El backend respondió HTTP 201 (Created), confirmando la creación del usuario y devolviendo su información con los campos permitidos, sin exponer contraseñas, hashes ni tokens. Finalmente, se realizó una solicitud de creación sin presentar un token JWT. La API respondió HTTP 401 (Unauthorized), impidiendo la operación sin autenticación. En comparación con la evaluación original, se confirmó que la creación de usuarios continúa utilizando correctamente HTTP 201 y que el acceso al endpoint ahora requiere autenticación JWT y permisos administrativos. La vulnerabilidad previamente observada, que permitía realizar operaciones administrativas sin JWT mediante el encabezado usuario-actual, no se reprodujo durante el retest.	PASS	Evidencias
29	**MEDIA**	G2-26	API / HTTP	Solicitar un recurso inexistente	Devuelve 404	Se verificó el comportamiento de la API al solicitar un recurso inexistente mediante los endpoints GET y PATCH /api/v1/users/{usuario_id}, utilizando el ID 999999 y autenticación JWT de administrador. Inicialmente, se ejecutó una solicitud GET /api/v1/users/999999. El backend respondió HTTP 404 (Not Found), con el mensaje "Usuario no encontrado", confirmando que identifica correctamente la ausencia del recurso solicitado. Posteriormente, se realizó una solicitud PATCH /api/v1/users/999999 enviando un cuerpo JSON vacío ({}). La API respondió nuevamente HTTP 404 (Not Found), rechazando la modificación del usuario inexistente.En comparación con la evaluación original, se confirmó que actualmente está disponible el endpoint GET para consultar usuarios individualmente. Anteriormente, esta solicitud devolvía HTTP 405 (Method Not Allowed), lo que impedía completar la prueba mediante GET. Durante el retest, ambos métodos respondieron con el código HTTP esperado.	PASS	Evidencias

30	**MEDIA**	G2-27	API / Estados	Intentar crear un usuario con un correo ya registrado	Devuelve 409 y el usuario duplicado no se crea	Se confirmó mediante GET /api/v1/me que la solicitud se realizó con una cuenta administradora autorizada. Posteriormente, se ejecutó POST /api/v1/users intentando crear un usuario con el correo previamente registrado qa.revisor@qa.udv.edu.gt. La API respondió HTTP 409 (Conflict), con código de error CONFLICT y mensaje "El correo ya está registrado". Adicionalmente, se realizó una consulta de solo lectura a MySQL sobre la tabla usuario, confirmando que existe únicamente un registro asociado al correo evaluado (ID 12). Por tanto, se verificó que la API rechazó la solicitud y no generó un usuario duplicado.	PASS	Evidencias
31	**MEDIA**	G2-28	API / Validación	Provocar un error de validación	Devuelve 422	Se ejecutó una solicitud POST /api/v1/users con una cuenta administradora autorizada, enviando intencionalmente un valor numérico (12345) en el campo correo, que requiere una cadena de texto. La API respondió HTTP 422 (Unprocessable Content), con código VALIDATION_ERROR y mensaje "Datos de entrada inválidos". En el campo details.campos se identificó correctamente el atributo correo, con el mensaje "Input should be a valid string" y tipo de error string_type. Se confirmó que la API rechaza los datos de entrada inválidos mediante una respuesta JSON estructurada, sin completar la creación del usuario. El escenario cumple el resultado esperado con el mecanismo actual de autenticación JWT.	PASS	Evidencias
32	**MEDIA**	G2-29	API / JSON	Revisar la estructura de las respuestas	Respeta el formato JSON acordado	La API respondió HTTP 422 (Unprocessable Content), con encabezado Content-Type: application/json y una estructura de error uniforme que contiene los campos error, code, message y details. Se obtuvo el código VALIDATION_ERROR, el mensaje "Datos de entrada inválidos" y el detalle "JSON decode error", con tipo json_invalid. Se confirmó que el backend rechaza solicitudes con sintaxis JSON inválida y devuelve una respuesta estructurada sin exponer información sensible. El código principal de error cambió respecto a la prueba original, de JSON_INVALID a VALIDATION_ERROR, manteniendo la identificación específica del error mediante json_invalid.	PASS	Evidencia
33	**MEDIA**	G2-30	API / Errores	Revisar la estructura de las respuestas de error	Mantiene el formato común definido para errores	se realizaron pruebas funcionales en Swagger sobre los códigos HTTP 400 (BAD_REQUEST), 401 (UNAUTHORIZED), 403 (FORBIDDEN), 404 (NOT_FOUND) y 422 (VALIDATION_ERROR). Se confirmó que todas las respuestas evaluadas mantienen una estructura JSON uniforme con los campos error, code, message y details. Adicionalmente, se revisaron las funciones _payload() y error_response() del manejador de errores, verificando que utilizan el formato común actualizado. El escenario cumple el comportamiento esperado dentro del alcance probado; no se evaluaron todos los posibles errores del sistema.	PASS — Errores evaluados	Evidencias

34	**MEDIA**	G2-33	OpenAPI	Comparar endpoints documentados contra los endpoints reales	OpenAPI coincide con el comportamiento real del sistema	e revisó la documentación OpenAPI publicada en Swagger y se comparó con el comportamiento real de los endpoints POST /api/v1/users, GET /api/v1/me y GET /api/v1/users En POST /api/v1/users se confirmó que la documentación continúa omitiendo HTTP 409 (Conflict), aunque este código fue comprobado funcionalmente en G2-27. Asimismo, el ejemplo de respuesta HTTP 201 muestra un valor genérico de tipo string en lugar de representar el objeto JSON devuelto por la API. El ejemplo documentado para HTTP 422 tampoco coincide con la estructura uniforme de errores validada en G2-28 y G2-30. En GET /api/v1/me se verificó una corrección respecto a la evaluación original: HTTP 401 ya aparece documentado con una descripción correspondiente a los errores de autenticación. Sin embargo, el ejemplo HTTP 200 continúa mostrando un string en lugar del objeto JSON que contiene la información, roles y permisos del usuario autenticado.Finalmente, en GET /api/v1/users se observó que la documentación únicamente presenta HTTP 200, sin incluir HTTP 403 (Forbidden), respuesta previamente comprobada con un usuario sin permisos de administrador. Además, el ejemplo HTTP 200 muestra un string en lugar del arreglo JSON de usuarios devuelto por el endpoint.	PARCIAL — Persisten inconsistencias en OpenAPI.	Evidencias
35	**BAJA**	G2-31	Paginación	Solicitar un listado con varias páginas	page, per_page, total y data funcionan correctamente	Se verificó que el endpoint continúa mostrando "No parameters", sin ofrecer parámetros de paginación como page, per_page, limit u offset. Posteriormente, se ejecutó GET /api/v1/users con autenticación administrativa válida, obteniendo HTTP 200 OK. Sin embargo, la respuesta consiste directamente en un arreglo JSON de usuarios, sin incluir los campos page, per_page, total y data requeridos por el escenario. Se confirma que el endpoint permite consultar usuarios, pero no implementa la paginación esperada. El hallazgo original persiste y no está relacionado con problemas de autenticación JWT ni de permisos. El escenario no cumple el resultado esperado y requiere implementar parámetros de paginación, limitar los registros devueltos y proporcionar los metadatos correspondientes.	FAIL — Paginación no implementada	Evidencias

36	**BAJA**	G2-32	Fechas	Revisar las fechas devueltas por la API	Utilizan ISO 8601 y la zona horaria acordada	En la evaluación original se obtuvo un resultado PASS al identificar el campo creado_en con formato ISO 8601 en GET /api/v1/users. Durante el retest se comprobó que la respuesta actual de GET /api/v1/users ya no incluye campos de fecha. Se revisó la implementación de los endpoints GET /users y GET /users/{usuario_id}, confirmando que ambos utilizan la función diccionario() para construir sus respuestas. La revisión de diccionario() demostró que únicamente se devuelven los campos id, correo, nombres, apellidos, activo y roles. Asimismo, se verificó en models.py que el modelo Usuario conserva los campos creado_en, actualizado_en y otros atributos DateTime, pero estos no se incluyen en las respuestas evaluadas. Por tanto, se identificó una diferencia respecto a la respuesta documentada originalmente. No obstante, no se ha confirmado si la exclusión de estos campos corresponde a una modificación intencional del contrato de la API. Resultado: PASS histórico. Retest pendiente de validación, debido a que los endpoints actuales no exponen fechas que permitan comprobar su formato y zona horaria. No se confirma un defecto funcional.	PASS histórico — Retest pendiente de validación	Evidencias
37	**BAJA**	G2-34	Login / Seguridad	Realizar 5 intentos fallidos consecutivos de login	El usuario queda bloqueado durante 15 minutos	se evaluó el mecanismo de bloqueo temporal por intentos fallidos de inicio de sesión mediante Swagger, utilizando la cuenta QA qa.revisor@qa.udv.edu.gt (rol reviewer) Inicialmente, se verificó que la cuenta podía iniciar sesión correctamente mediante POST /api/v1/auth/login, obteniendo HTTP 200. Posteriormente, se realizaron cinco intentos consecutivos de inicio de sesión con una contraseña incorrecta. Los primeros cuatro intentos devolvieron HTTP 401 (UNAUTHORIZED), mientras que el quinto devolvió HTTP 429 (TOO_MANY_REQUESTS), con el mensaje "Cuenta bloqueada por intentos fallidos. Intenta de nuevo en 15 minutos". Durante el período de bloqueo, se intentó iniciar sesión utilizando la contraseña correcta. La API respondió nuevamente HTTP 429, indicando que la cuenta continuaba bloqueada y que faltaban aproximadamente 13 minutos para permitir un nuevo intento. Finalmente, después de transcurrido el período de bloqueo, se realizó un nuevo inicio de sesión con las credenciales correctas, obteniendo HTTP 200 y un nuevo access_token.	PASS	Evidencias

38	**BAJA**	G2-36	Auditoría	Realizar una acción sensible	El acceso o acción queda registrado según corresponda	se verificó funcionalmente el registro de auditoría mediante Swagger y consultas de solo lectura a MySQL.Inicialmente, se ejecutó POST /api/v1/auth/login con un correo inexistente, obteniendo HTTP 401. Posteriormente, se confirmó en la tabla auditoria la creación del evento LOGIN_FALLIDO (ID 74), con registro_id = 0, actor_id = NULL y el correo utilizado en el campo detalle. Esto confirma la corrección del problema reportado previamente, relacionado con la restricción NOT NULL de registro_id. Adicionalmente, se revisó la integración de auditoría en el código y se consultaron los eventos existentes en MySQL. Para validar una acción sensible, se actualizó temporalmente el nombre del usuario QA ID 12 mediante PATCH /api/v1/users/12, utilizando la cuenta administradora ID 9. La operación devolvió HTTP 200 y generó el registro de auditoría ID 75, con acción update, tabla usuario, registro_id = 12 y actor_id = 9. Finalmente, se restauró el nombre original del usuario de prueba mediante PATCH, obteniendo HTTP 200. Se confirma que los intentos fallidos de autenticación con correos inexistentes y las actualizaciones de usuarios evaluadas quedan registrados correctamente en la base de datos. El hallazgo original queda corregido dentro del alcance probado; no se evaluaron todas las posibles acciones sensibles del sistema.	PASS	Evidencias

Prioridad ALTA: validar primero por impacto en seguridad y funcionamiento.
Prioridad MEDIA: validar contrato, comportamiento e integración de la API.
Prioridad BAJA: validaciones complementarias que igualmente deben revisarse antes del cierre.